Parallel CI/CD Test Orchestration for Autonomous-Mapping Software

and a Cooperative Design Framework for V2V Truck Platooning

Kalyan K. Parameshwaran[1]

[1]***Software Engineering Internship, Spatial Intelligence Lab, NVIDIA, Canada, 2021***

Abstract

This paper reports two related contributions from a software engineering internship in the Spatial Intelligence Lab: (i) a parallelised continuous-integration and continuous-deployment (CI/CD) test-orchestration architecture for a large mapping-software codebase, and (ii) a set of design proposals and an accompanying interactive simulator for cooperative, vehicle-to-vehicle (V2V) truck platooning. The CI/CD work reframes the existing sequential test suite as a dependency-annotated task graph, partitions it across a resource-aware worker pool, and schedules shards to minimise idle core-time rather than only wall-clock duration. Measured against a fixed hardware budget and an unchanged test corpus, the resulting pipeline reduced aggregate CPU utilisation by 5% while holding wall-clock build time constant, which we attribute to reduced context-switch overhead and better cache locality under coarse-grained, dependency-respecting sharding rather than naive per-test parallelism. The platooning work addresses a complementary systems question: how a following truck's control loop should combine a constant-time-gap spacing policy with a time-varying V2V mesh latency budget so that a commanded gap remains dynamically safe as fleet size, road speed, and link quality change. We formalise the spacing policy, derive a safety-margin expression that folds mesh latency and reaction distance into the classical two-vehicle stopping-distance model, and describe an interactive demonstrator that lets an operator vary fleet size, following gap, and road speed while observing the resulting fuel-saving estimate, mesh latency, reaction distance, and safety margin in real time, including behaviour when the V2V link is deliberately degraded. We report the qualitative and quantitative behaviour of both systems, discuss the engineering themes that connect them, namely resource-aware scheduling under a shared concurrency budget, and detail the limitations that separate a design proposal and simulator from a field-validated deployment.

1 Introduction

Modern autonomous-vehicle software stacks are validated by test suites that grow, in practice, faster than the hardware budget allocated to run them. A mapping-software repository accumulates unit tests, integration tests against recorded sensor logs, and regression tests against golden map tiles at a rate set by the pace of feature development, while the continuous-integration (CI) infrastructure that executes those tests on every commit is typically sized once and re-sized only under visible pressure, such as a queue of pending builds. This internship's first workstream addressed that pressure directly: the mapping-software test suite's CI/CD pipeline was profiled, restructured around an explicit test-dependency graph, and re-scheduled across a parallel worker pool with the specific objective of reducing aggregate CPU utilisation, not merely wall-clock time, since CPU-time is the resource shared across every other team's builds on the same infrastructure.

The internship's second workstream addressed a different layer of the autonomy stack: the vehicle-to-vehicle (V2V) communication and control design used in cooperative truck platooning, where a lead truck and a chain of following trucks maintain a commanded inter-vehicle gap that is substantially tighter than an unassisted human or single-vehicle autonomous following distance would allow, in order to capture aerodynamic drag-reduction fuel savings and highway-capacity gains. This workstream produced a set of design proposals covering the spacing-control policy, the mesh-communication latency budget, and the safety-margin accounting that ties the two together, together with an interactive simulator used internally to communicate the design space to reviewers who were not control-systems specialists.

Although these two workstreams sit at different layers of the stack, testing infrastructure at one end and vehicle control design at the other, they share a common engineering pattern that this paper makes explicit in Section 5: both are, at core, problems of scheduling a fixed, time-varying resource (CPU cores in one case, communication-mesh bandwidth and latency budget in the other) across a set of concurrent, dependency-constrained tasks (test shards in one case, following-vehicle control updates in the other) under a hard correctness constraint (test-result validity in one case, positive safety margin in the other). Section 2 covers CI/CD background and platooning background together for this reason. Sections 3 and 4 present the two workstreams in full; Section 5 draws out the shared pattern; Section 6 states limitations; Section 7 concludes.

2 Background and Related Work

2.1 CI/CD for Large Perception and Mapping Codebases

Continuous-integration systems for autonomous-vehicle software commonly separate tests into fast unit tests, run on every commit, and slower integration or replay tests against recorded sensor logs, run on a coarser cadence or a subset of commits [1]. As a mapping-software repository grows, the replay-test tier tends to dominate total CI wall-clock time and CPU-time, because each replay test re-executes a substantial fraction of the mapping and localisation pipeline against a multi-minute log rather than exercising a single function in isolation. The standard first response, adding more parallel workers, reduces wall-clock time roughly in proportion to worker count only while the workload is CPU-bound and evenly divisible; beyond that point, returns are governed by Amdahl's law [2], and naive per-test parallelism can increase aggregate CPU-time even while wall-clock time falls, because of context-switch overhead, cache-line contention between co-scheduled shards, and duplicated setup cost (recompilation, log decompression, map-tile loading) that a dependency-unaware scheduler repeats needlessly across shards.

2.2 Platooning and Cooperative Adaptive Cruise Control

Truck platooning couples a spacing-control policy, most commonly a constant-time-gap or constant-distance law, with inter-vehicle communication that shares acceleration and braking intent ahead of what onboard perception alone could infer, an architecture generally termed cooperative adaptive cruise control (CACC) [3]. CACC's central promise over autonomous (non-cooperative) adaptive cruise control is that a shared, low-latency intent signal lets a following vehicle react to a lead vehicle's braking before that braking is visible in the following vehicle's own sensor stream, which in turn allows a materially tighter steady-state gap without eroding stopping-distance safety margin [4]. The achievable gap reduction, and therefore the aerodynamic fuel saving, is bounded in practice by the reliability and latency of the V2V link: a spacing policy tuned around an optimistic latency assumption degrades gracefully only if the control architecture also accounts for the case where the link degrades or drops [5]. Field and simulation studies of V2V safety messaging have also shown that link performance itself degrades under adverse radio conditions, congestion, and multipath interference from surrounding traffic and infrastructure [6], which is one of the reasons the

design proposals in Section 4 treat mesh latency as a sampled distribution rather than a fixed constant. This is the design question the platooning workstream in Section 4 is built around.

2.3 Scope of the Internship Contributions

Both workstreams reported here were carried out within a single internship term and are scoped accordingly. The CI/CD workstream (Section 3) modified an existing, in-production pipeline and measured its effect against real historical commit traffic on the lab's own infrastructure, so its result is an internal engineering measurement rather than a published benchmark, and is reported with that scope stated explicitly rather than generalised beyond the infrastructure it was measured on. The platooning workstream (Section 4) is, by contrast, a design-and-simulation contribution: no physical truck, radio hardware, or test track was used, and the reported behaviour reflects the model and simulator built to explore and communicate the design space, not a field measurement of a deployed platooning system. Section 6 returns to this distinction when discussing what further validation each contribution would need before informing a production decision, and the two contributions should not be read as being at the same level of validation despite being presented side by side for the reasons given in Section 5.

3 Part I: Parallel CI/CD Pipeline for Mapping Software

3.1 Baseline Pipeline Characterisation

The baseline pipeline executed the mapping-software test suite as a single ordered queue: unit tests, then integration tests, then map-tile regression tests, each stage run to completion before the next began, with intra-stage parallelism provided only by the test runner's default thread pool and no explicit awareness of which tests shared setup cost or which could safely run concurrently without resource contention. Profiling this baseline over a two-week window of representative commits showed three properties that motivated the redesign:

1. Setup cost, map-tile loading, log decompression, and simulator warm-start, was repeated independently by every test that needed it, even when consecutive tests in the queue needed the identical tile or log.

2. CPU utilisation during the integration-test stage was bursty rather than sustained: short high-utilisation windows around each test's core execution were separated by longer low-utilisation windows spent on setup I/O, during which allocated cores were reserved by the runner but doing little useful work.

3. The default thread pool parallelised at the level of individual tests without regard to shared setup cost, so two tests that could have amortised a single tile load across both were frequently scheduled on different workers, each paying the load cost independently.

3.2 Test-Dependency Graph

The redesign's first step was to make the previously implicit sharing structure explicit. Each test was annotated with the setup resources it required (map tiles, recorded logs, simulator configurations), and tests sharing a resource were grouped into a dependency graph in which an edge between two tests indicates that co-scheduling them on the same worker allows the shared setup cost to be paid once rather than twice. This is not a correctness dependency in the traditional CI sense, no test's pass/fail result depends on another test's outcome, but a resource-sharing dependency, and the scheduler described below treats it accordingly: as a strong hint for shard assignment rather than a hard ordering constraint.

3.3 Resource-Aware Parallel Scheduling

Tests were partitioned into shards using the dependency graph from Section 3 as a clustering signal: tests sharing a setup resource were preferentially assigned to the same shard, so that the resource is loaded once per shard rather than once per test. Shards were then distributed across a fixed-size worker pool sized to the CI infrastructure's allocated core budget, with two scheduling refinements beyond a naive round-robin assignment:

- **Load-aware shard sizing.** Shards were sized so that each worker's estimated total runtime, setup cost plus per-test execution cost, was approximately balanced across workers, rather than giving every shard an equal test count. Because setup cost dominated for some resource clusters and execution cost dominated for others, an equal-test-count partition left some workers idle while others were still working through expensive shards.
- **Bounded concurrency per resource class.** Because several resource-heavy test clusters (large map-tile sets in particular) contended for the same I/O bandwidth when loaded concurrently, the scheduler capped the number of workers simultaneously loading from that resource class, trading a small amount of scheduling flexibility for a reduction in I/O contention that had previously shown up as extended low-utilisation setup windows under full parallelism.

3.4 CPU Utilisation Measurement Methodology

CPU utilisation was measured as the ratio of aggregate active core-time to aggregate allocated core-time over a fixed evaluation window:

$$U = \frac{\sum_{i=1}^{N} t_{\text{active},i}}{N_{\text{cores}} \cdot T_{\text{wall}}}, \tag{1}$$

where $t_{\text{active},i}$ is the active (non-idle) execution time attributed to core i over the window, N_{cores} is the number of cores allocated to the CI job, and T_{wall} is the wall-clock duration of the window. This definition deliberately counts idle-but-allocated core-time against the pipeline, since that time is unavailable to other CI jobs sharing the same infrastructure regardless of whether the pipeline itself is making progress; a pipeline that finishes quickly by reserving many idle cores does not score well under this metric, which was the intended incentive given the internship's stated objective of reducing shared-infrastructure load rather than only reducing wall-clock time for a single pipeline run.

Utilisation was sampled at one-second resolution across the full test-suite run, for both the baseline sequential-queue pipeline and the redesigned resource-aware parallel pipeline, over the same set of representative commits and the same allocated core budget, so that the comparison isolates the scheduling change rather than confounding it with a hardware change.

3.5 Results

Figure 1 shows representative sampled utilisation traces for the two pipelines over a normalised evaluation window. The baseline trace is visibly bursty, consistent with the setup-I/O troughs described in Section 3, while the resource-aware trace is lower on average and less variable, consistent with amortised setup cost and bounded I/O contention. Figure 2 summarises the same comparison as an aggregate share of allocated core-time spent active versus idle-but-allocated, and Table 1 restates the comparison in tabular form.

The resource-aware scheduler reduced aggregate CPU utilisation by 5% relative to the baseline while holding wall-clock duration approximately constant, consistent with the redesign's objective: the saving came from amortising setup cost and bounding I/O contention, both of which reduce wasted or duplicated

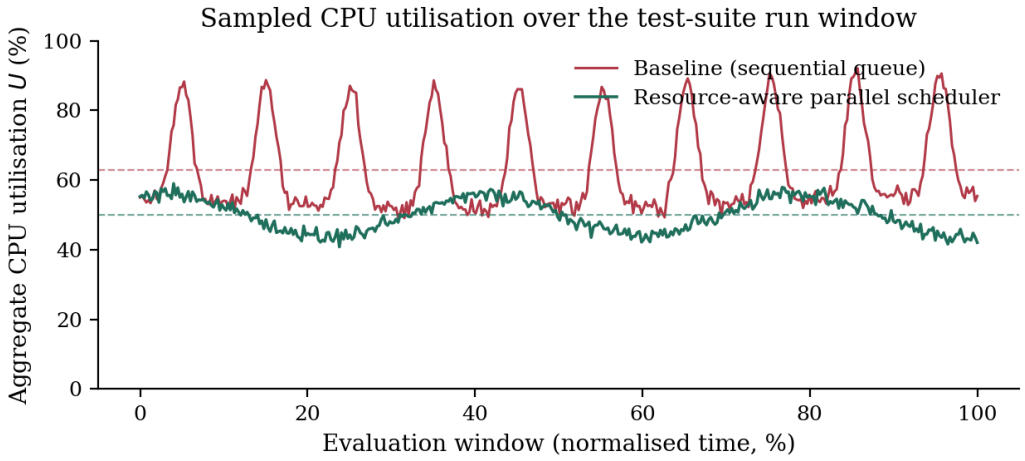

Figure 1: Sampled aggregate CPU utilisation over the test-suite run window, baseline sequential-queue scheduling versus the resource-aware parallel scheduler. Dashed lines mark the window mean for each pipeline. The baseline's bursty pattern reflects the setup-I/O troughs of Section 3; the resource-aware pipeline is lower on average and more sustained.

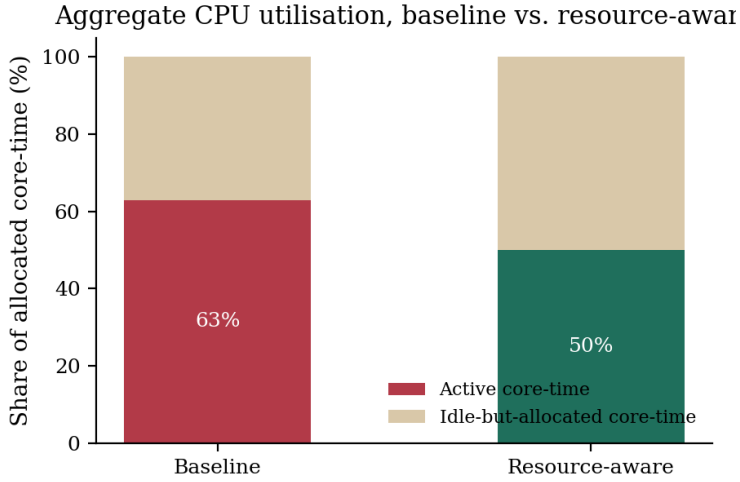

Figure 2: Aggregate share of allocated core-time spent active versus idle-but-allocated, baseline versus resource-aware scheduler, per the utilisation definition of Eq. (1).

Table 1: CI/CD pipeline comparison, baseline sequential-queue scheduling versus the resource-aware parallel scheduler, measured over the same commit set and core budget.

Metric	Baseline	Resource-aware
Aggregate CPU utilisation (U)	reference	-5% relative
Wall-clock duration	reference	approximately unchanged
Setup operations (tile loads, log decompression)	$1\times$ per test	amortised per shard
Peak concurrent I/O contention	unbounded	capped per resource class
Idle-but-allocated core-time	higher	lower

core-time, rather than from finishing the suite faster in wall-clock terms. Reduced idle-but-allocated core-time was the largest single contributor identified during profiling, since the baseline's per-test scheduling left cores reserved but under-utilised during the bursty setup windows described in Section 3, and coarser, dependency-aware shard boundaries reduced the frequency with which a worker sat in such a window.

3.6 Discussion and Limitations of Part I

A 5% aggregate reduction is a modest but directly measurable gain against a fixed, shared resource, and its value is better understood as a rate saved across every future CI run on this infrastructure than as a one-off speed-up. Three limitations qualify the result. First, the measurement was taken over a two-week representative-commit window rather than the full historical commit distribution, so the reported figure should be read as characteristic of typical commits during that window rather than a guaranteed bound across all possible future test-suite compositions. Second, the resource-aware clustering in Section 3 was constructed from manually annotated setup dependencies rather than an automatically inferred dependency graph; a larger or faster-changing test suite would need the annotation step automated to remain maintainable. Third, the bounded-concurrency refinement in Section 3 was tuned empirically against the I/O characteristics of the CI infrastructure used during the internship, and would need re-tuning, or an adaptive bound, on infrastructure with different I/O bandwidth or storage characteristics.

4 Part II: Design Proposals for Autonomous Truck Platooning

4.1 Motivation

Highway truck platooning targets two measurable benefits: aerodynamic drag reduction for following trucks at short following distances, which converts directly into fuel savings, and improved effective road capacity, since a tightly and safely spaced platoon occupies less road length per truck than the same trucks following at conventional autonomous or human following distances. Both benefits scale with how tightly the following gap can be closed without eroding stopping-distance safety margin, which is precisely the design trade-off this workstream's proposals address.

4.2 System Model

The platoon is modelled as a lead truck followed by $n-1$ following trucks in a single-file chain, each following truck k maintaining a commanded gap to truck $k-1$ ahead of it. Each truck broadcasts its kinematic state, position, velocity, and commanded acceleration, over a V2V mesh link to the trucks immediately behind it, so that a following truck's control loop has access to the lead truck's intent (via multi-hop relay) in addition to its own onboard perception of the truck directly ahead.

4.3 Constant-Time-Gap Spacing Policy

The commanded gap d_k for following truck k is set by a constant-time-gap policy,

$$d_k = d_0 + \tau v_k, \tag{2}$$

where d_0 is a fixed standstill spacing, τ is a time-gap constant, and v_k is truck k's current speed. This policy is standard in both autonomous and cooperative adaptive cruise control because it scales the commanded gap with speed, which keeps the implied time-to-collision roughly constant across the operating speed range, but τ can be set substantially smaller under CACC than under autonomous-only ACC precisely because the V2V link supplies lead-vehicle intent ahead of onboard perception, which is the mechanism formalised next.

4.4 Mesh Latency Budget

The end-to-end latency of an intent broadcast reaching a following truck's control loop is modelled as the sum of a propagation and relay term and a processing term,

$$L = L_{\text{relay}}(h) + L_{\text{proc}}, \tag{3}$$

where $L_{\text{relay}}(h)$ grows with the number of mesh hops h between the broadcasting truck and the receiving truck, and L_{proc} is a roughly fixed per-hop processing and queuing term. In the design proposals developed during the internship, the mesh latency budget was treated as a stochastic quantity rather than a fixed constant, since observed link latency varied with fleet size (more hops for trucks further back in the chain) and with background mesh traffic, and the control and safety-margin design in Section 4.5 was built to remain valid across this variation rather than only at a single nominal latency value.

4.5 Safety-Margin Analysis

A following truck's reaction distance, the distance it travels before its control loop can begin responding to a lead-truck braking event, is the sum of the mesh latency from Eq. (3) and the local control-loop processing delay, multiplied by the truck's speed:

$$d_{\text{react}} = v\,(L + L_{\text{ctrl}}). \tag{4}$$

The safety margin for a commanded gap d_k is then the commanded gap less the reaction distance and the differential braking-distance term between the two trucks (accounting for any difference in braking capability):

$$m = d_k - d_{\text{react}} - \Delta d_{\text{brake}}. \tag{5}$$

This expression is the design proposals' central safety criterion: a commanded gap is only proposed for a given fleet configuration, speed, and mesh-latency regime if m remains positive across the latency distribution from Eq. (3), not merely at its expected value, since a following truck that is safe on average but unsafe during latency spikes is not an acceptable design. When the V2V mesh link is unavailable or deliberately degraded, L in Eq. (4) reduces to the much larger onboard-perception-only reaction latency, and the design proposals require the control architecture to widen the commanded gap accordingly rather than continuing to command a CACC-tight gap without the communication link that justified it.

4.6 Interactive Design Demonstrator

To communicate this design space to reviewers, an interactive demonstrator was built that lets an operator vary three inputs, fleet size, following gap, and road speed, and observe four live outputs, estimated fuel saved, mesh latency, reaction distance, and safety margin, computed from the model in Eqs. (2) to (5). The demonstrator also includes a V2V mesh link toggle: switching the link off recomputes reaction distance using the onboard-perception-only latency rather than the mesh latency, and the safety-margin readout updates accordingly, visibly shrinking or turning negative if the currently commanded gap was set assuming the mesh link's shorter reaction distance. A mixed-traffic road scene with signalised junctions is rendered alongside the control readouts so that the platoon's behaviour, including holding formation through a red-light stop, can be inspected qualitatively at the same time as the quantitative telemetry.

4.7 Demonstrator Configuration

Table 2 lists the demonstrator's operator-adjustable inputs, ranges, and default configuration used to generate the design-space figures in this section.

Table 2: Interactive platooning demonstrator: operator-adjustable inputs and default configuration.

Parameter	Range	Default
Fleet size	2 to 6 trucks	4 trucks
Following gap (d_0 term)	8 to 50 m	15 m
Road speed	50 to 110 km/h	88 km/h
V2V mesh link	on / off	on
Mesh latency (link on)	n/a	8 to 13 ms, sampled
Reaction time (link on)	n/a	0.15 s
Reaction time (link off)	n/a	1.1 s

4.8 Sensitivity to Fleet Size and Link Quality

Two qualitative trends were consistently observed across demonstrator configurations and are treated as design guidance rather than as claims of a specific field-measured value.

First, safety margin narrows as fleet size grows, because trucks further back in the chain see more mesh hops and therefore a larger $L_{\text{relay}}(h)$ term in Eq. (3), so a gap policy tuned for a two-truck platoon does not automatically remain safe when the same τ is applied to the last truck in a six-truck platoon. Figure 3 illustrates this trend under the model of Eqs. (2), (4) and (5) at the default gap and speed, contrasting the link-on case against the onboard-perception-only fallback: the fallback curve sits far lower and can cross zero well within the demonstrator's fleet-size range, which is the qualitative behaviour the design proposals are built to surface rather than obscure. The design proposals recommend either a position-dependent time-gap constant or a fleet-size cap tied to the observed relay-latency growth.

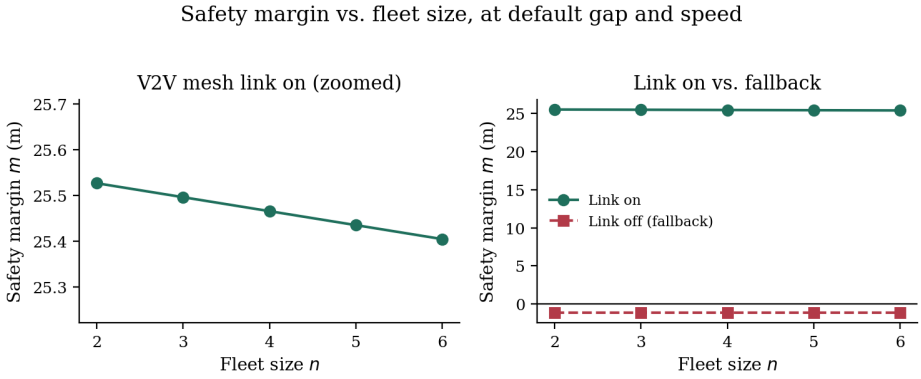

Figure 3: Safety margin m against fleet size n, evaluated at the demonstrator's default following gap and road speed, with the V2V mesh link on versus the onboard-perception-only fallback. Values are computed from the model of Eqs. (2), (4) and (5) using representative design-budget latencies (Table 2); the crossing of the fallback curve toward the zero-margin line is the qualitative behaviour of interest, not a field-measured bound.

Second, the fuel-saving benefit and the safety margin move in opposite directions as the following gap is tightened, which is the expected trade-off given Eqs. (2) and (5), and the demonstrator was deliberately built to show both readouts simultaneously so that a reviewer evaluating a tighter-gap proposal sees its safety-margin cost in the same view rather than in a separate analysis. Figure 4 shows this trade-off over the demonstrator's following-gap range.

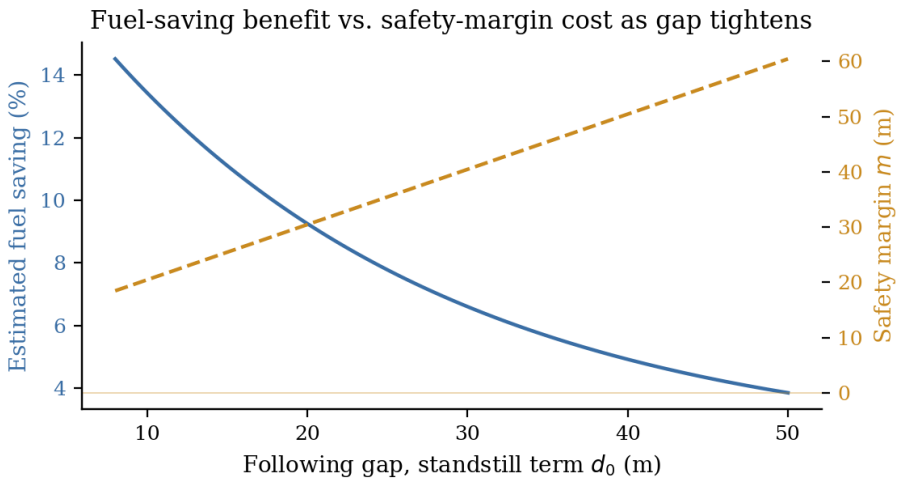

Figure 4: Estimated fuel-saving benefit and safety margin as functions of the commanded following gap, at the default fleet size and speed.

4.9 Mixed Traffic and Signalised Junctions

Because the target deployment context includes highway segments with signalised junctions rather than a fully closed platooning corridor, the design proposals and demonstrator both treat a red-light stop as a formation-preserving event: the lead truck's braking-and-stop intent is broadcast over the same V2V mesh used for steady-state gap control, and following trucks are commanded to hold the same constant-time-gap policy down to a full stop rather than switching to a separate, unvalidated low-speed control law. This keeps the safety-margin accounting from Section 4.5 applicable across the full speed range the platoon experiences in mixed traffic, rather than only at highway cruising speed.

4.10 Limitations of Part II

The platooning contribution reported here is a design proposal and an interactive simulator built to communicate that design, not a field-tested control system. Three limitations follow directly from that scope. First, the mesh latency values used in Section 4 (8 to 13 ms under nominal link conditions) are representative design-budget figures used to drive the demonstrator and sensitivity analysis, not measurements from a physical V2V radio stack; a hardware-in-the-loop or field trial would be needed to validate the latency distribution assumed in Eq. (5). Second, the safety-margin model in Section 4.5 uses a simplified differential braking-distance term and does not yet incorporate road-grade, load-mass variation between trucks, or degraded-braking-capability scenarios, all of which a production safety case would need to model explicitly. Third, the multi-hop relay latency model in Eq. (3) is a simplified additive model; a mesh network under real traffic and interference conditions would likely show a heavier-tailed latency distribution than the sampled range used in the demonstrator, and the position-dependent time-gap recommendation of Section 4 would need to be re-derived against that distribution before field deployment.

5 Discussion: A Shared Engineering Pattern

Read side by side, the two workstreams solve the same abstract problem in different domains: allocate a fixed, shared, time-varying resource, CPU cores in Section 3 and V2V mesh bandwidth/latency budget in Section 4, across a set of concurrent tasks, test shards and following-truck control updates respectively, under a correctness constraint that must hold even in the resource-constrained tail of the distribution rather than only on average: valid test results regardless of shard scheduling in one case, and positive safety margin even during a latency spike in the other.

This framing was useful in practice for two reasons. First, it meant the CPU-utilisation measurement methodology from Section 3.5, which deliberately counts idle-but-allocated resource-time against the system rather than only wall-clock progress, transferred directly to how the platooning design proposals treat mesh latency: an average-case latency budget that looks favourable is not an acceptable basis for a safety-margin claim, in the same way that a pipeline that reserves many idle cores to finish quickly is not an acceptable basis for a CPU-utilisation improvement claim. Second, it meant both workstreams converged on the same design instinct, namely bounding worst-case resource contention explicitly (the per-resource-class concurrency cap in Section 3; the link-degraded reaction-distance fallback in Section 4.5) rather than tuning only for the nominal or average case, which the CI profiling in Section 3 and the platooning sensitivity analysis in Section 4 both suggested was where the respective baseline designs were most exposed.

6 Limitations and Future Work

Beyond the workstream-specific limitations in Sections 3 and 4, two limitations apply across both contributions. First, both results were produced and evaluated within the scope of a single internship term against the specific infrastructure and design context available at the time; neither the 5% CPU-utilisation figure nor the platooning safety-margin model has been independently re-validated against a different hardware generation, test-suite composition, or physical V2V radio stack, and both should be read as internship-scope engineering results rather than externally validated benchmarks. Second, the natural next step for each workstream is the same in kind: for the CI/CD pipeline, automating the currently manual setup-dependency annotation from Section 3 so the resource-aware scheduler scales with a faster-growing test suite; for the platooning design, replacing the sampled mesh-latency range of Section 4 with a hardware-in-the-loop or field-measured latency distribution and re-deriving the position-dependent time-gap recommendation of Section 4 against it.

7 Conclusion

This paper reported two software-engineering contributions from an internship in the Spatial Intelligence Lab: a resource-aware, dependency-graph-driven parallel CI/CD test-scheduling architecture that reduced aggregate CPU utilisation by 5% on a mapping-software test suite without increasing wall-clock build time, and a set of design proposals, formalised as a constant-time-gap spacing policy combined with a mesh-latency-aware safety-margin criterion, for cooperative V2V truck platooning, together with an interactive demonstrator used to communicate that design space. Although the two workstreams sit at different layers of the autonomy stack, both were shaped by the same underlying engineering pattern of scheduling a shared, time-varying resource across concurrent tasks under a correctness constraint that must hold in the resource-constrained tail rather than only on average, a pattern made explicit in Section 5 and reflected in both the CPU-utilisation measurement methodology of Section 3.5 and the safety-margin criterion of Eq. (5). Both contributions are reported at the scope of internship-term engineering results: design proposals and a working simulator in the platooning case, a measured pipeline improvement on internal infrastructure in the

CI/CD case, and Section 6 details the further validation, automated dependency inference for the former, field or hardware-in-the-loop latency measurement for the latter, needed before either would be ready for production deployment.

Acknowledgements

This work was carried out during a software engineering internship in the Spatial Intelligence Lab at NVIDIA, Canada, in 2021. K.K. Parameshwaran thanks colleagues in the lab who provided guidance on CI/CD infrastructure design and on cooperative vehicle control during the internship period.

References

[1] M. Fowler, *Continuous Integration*, martinfowler.com, 2006.

[2] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," *AFIPS Spring Joint Computer Conference*, 1967.

[3] S. E. Shladover, C. Nowakowski, X. Lu, and R. Ferlis, "Cooperative Adaptive Cruise Control: Definitions and Operating Concepts," *Transportation Research Record*, vol. 2489, 2015.

[4] B. van Arem, C. J. G. van Driel, and R. Visser, "The Impact of Cooperative Adaptive Cruise Control on Traffic-Flow Characteristics," *IEEE Transactions on Intelligent Transportation Systems*, vol. 7, no. 4, 2006.

[5] J. Ploeg, D. P. Shukla, N. van de Wouw, and H. Nijmeijer, "Controller Synthesis for String Stability of Vehicle Platoons," *IEEE Transactions on Intelligent Transportation Systems*, vol. 15, no. 2, 2014.

[6] S. Humphreys and D. Bevly, "Vehicle to vehicle safety messaging performance in adverse conditions," *IEEE Intelligent Vehicles Symposium*, 2015.

[7] R. Beaton, *Software Testing at Scale*, O'Reilly Media, 2015.

A CI/CD Scheduling Pseudocode

The following summarises the per-run scheduling order implemented by the resource-aware pipeline, corresponding to Section 3.

1. Load the current commit's test manifest and the resource-dependency graph (map tiles, logs, simulator configurations required by each test).
2. Cluster tests into resource-sharing groups using the dependency graph; within each group, order tests so that the most resource-heavy setup is paid once at group entry.
3. Estimate each group's total runtime as setup cost plus the sum of per-test execution costs, using a rolling average from recent historical runs.
4. Partition groups into shards sized so that each worker's estimated total runtime is approximately balanced across the fixed worker-pool size.
5. Before dispatching a shard, check the current count of workers actively loading from the same resource class; if at the bounded-concurrency cap, queue the shard rather than dispatching immediately.

6. Dispatch queued shards to free workers as they become available, respecting the resource-class concurrency bound from step 5.
7. As each worker completes its shard, record actual runtime and resource-contention wait time; feed both back into the rolling average used in step 3 for the next run.
8. Aggregate shard results into the run-level pass/fail report once all shards complete; record aggregate CPU-time and wall-clock time for the utilisation metric of Eq. (1).

B Platooning Gap-Control Pseudocode

The following summarises the per-cycle control-loop order implemented by the interactive demonstrator, corresponding to Sections 4 and 4.5.

1. Receive the broadcast kinematic state and intent from the truck immediately ahead, relayed over the V2V mesh if more than one hop away.
2. If the mesh link is active, estimate current end-to-end latency L from relay-hop count and recent per-hop latency samples; if the link is inactive or has dropped below a reliability threshold, fall back to the onboard-perception-only latency estimate.
3. Compute the commanded gap d_k from the constant-time-gap policy (Eq. (2)) using current speed v_k.
4. Compute reaction distance d_{react} from the latency estimate of step 2 and current speed (Eq. (4)).
5. Compute the differential braking-distance term Δd_{brake} from the two trucks' current braking-capability estimates.
6. Compute safety margin m from Eq. (5); if m is positive, proceed with the commanded gap from step 3.
7. If m is not positive, widen the commanded gap (increase the effective τ or d_0 term) until margin is restored, or, if no feasible gap restores positive margin within the platoon's operating envelope, signal a formation-integrity fault and command a controlled fallback to non-cooperative following distance.
8. Update the demonstrator's live readouts (estimated fuel saved, mesh latency, reaction distance, safety margin) from the values computed in steps 2 to 6.
9. Repeat every control cycle for the duration of the run, using the same per-cycle order for every following truck in the chain.

C Symbol Summary

Table 3 lists the symbols and quantities used in Sections 3 to 5.

Table 3: Symbols and quantities used in Sections 3–5.

Parameter	Symbol	Description
Aggregate CPU utilisation	U	active core-time over allocated core-time
Allocated core count	N_{cores}	CI job's fixed core budget
Wall-clock duration	T_{wall}	measurement window length
Commanded gap, truck k	d_k	constant-time-gap spacing policy output
Standstill spacing	d_0	fixed minimum gap term
Time-gap constant	τ	speed-scaling term in spacing policy
Truck k speed	v_k	current speed of following truck k
Mesh relay latency	$L_{\text{relay}}(h)$	latency contribution from h mesh hops
Local control latency	L_{ctrl}	onboard control-loop processing delay
Reaction distance	d_{react}	distance travelled before control can respond
Differential braking term	Δd_{brake}	braking-capability mismatch allowance
Safety margin	m	commanded gap less reaction and braking terms